Ders 6 Genişletilmiş Notu: Bilgi Erişimi ve Web Araması: Boolean Model, Vektör Uzayı Modeli ve TF-IDF

Ders 6 Genişletilmiş Notu: Bilgi Erişimi ve Web Araması: Boolean Model, Vektör Uzayı Modeli ve TF-IDF

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Web İçerik Madenciliğinin Kapsamı
- 2. Bilgi Erişimi ile Veri Tabanı Sorgusu Arasındaki Fark
- 3. Arama Motorları ve Sonuçtan Bilgeliğe
- 4. Web Araması: Bilgi Erişiminin Özel Bir Hali
- 5. Web Arama Motorlarının Çözmesi Gereken Problemler
- 6. Bilgi Erişimi Modelinin Bileşenleri
- 7. Sistem Mimarisi ve Çalıştırılabilir Sorgu
- 8. Sorgu Dili Örnekleri: Dijital Kütüphane Arayüzleri
- 9. Sorgu Tipleri
- 10. Boolean Bilgi Erişimi Modeli
- 11. Vektör Uzayı Modeli ve TF-IDF
- 12. Sıralama Fonksiyonu: Kosinüs ve Okapi Benzerliği
- 13. İlgililik Geri Bildirimi
- 14. Dönem Projesi Sunumları

1. Web İçerik Madenciliğinin Kapsamı

Bu derste kitabın 6. bölümüne başlandı. Daha önce 1. bölümde genel bir bakış yapılmış, ardından 2., 3. ve 4. bölümler işlenmişti. Bu ders ise web madenciliğinin birinci kategorisi olan web içerik madenciliğine (web content mining) ayrılmıştır.

1.1 Fiziksel Ağ ve Web

Hoca önce dersin başında söylenen bir noktayı hatırlattı. Web'deki dokümanlar, dağıtık ortamda duran uygulama sunucuları (application server) üzerindeki HTML dokümanlarından oluşur. Bu dokümanların içinde köprü bağlantıları (hyperlink) vardır. Dokümanlar bu bağlantılarla birbirine link verir ve web dediğimiz ağı oluşturur.

Bu ağa bir doküman ağı denebilir. Burada insanlar herhangi bir otoriteye bağlı kalmadan birbirine link verir. Bir sayfaya link vermek, o sayfaya oy vermek ve ona önem atfetmek gibidir.

Hoca iki katmanı birbirinden ayırdı:

- **Fiziksel ağ (internet):** Bağlantıları fiber optik kablolar ve Wi-Fi, Bluetooth gibi kablosuz veri aktarım yöntemleri oluşturur. İnternete bağlanan farklı cihazlar bu sayede birbiriyle konuşur.

- **Web:** Fiziksel ağın birbirine bağladığı sunucuların üzerinde kuruludur. Özellikle IIS, Tomcat ve HTTP sunucuları gibi uygulama sunucularında duran, HTML ile yazılmış dokümanların birbirine verdiği linklerden oluşan sanal bir ağdır.

1.2 Toplamadan Sıralamaya

Dokümanlar farklı sunucularda olduğu için arama yapılabilmesi adına bir dizi iş gerekir. Önce dokümanlar merkezi bir ortamda toplanır, yani taranır (crawl edilir). Ardından üzerinde arama yapılabilecek şekilde indekslenir, yani bir indeks tablosu oluşturulur. Kullanıcının girdiği kelime ya da kelime grupları bu tabloda aranır. Eşleşen sonuçlar belirli bir sıralama yöntemine göre kullanıcıya gösterilir.

Web içerik madenciliği, özellikle tarama bittikten sonra başlayan kısmı kapsar. Dağıtık ortamdaki HTML dokümanları merkezde toplandıktan sonra, üzerlerinde arama yapılana kadarki bütün işler buna dahildir:

- Dokümanları toplamak
- Dokümanları ayrıştırmak (parse)
- İçlerinden kelimeleri çıkarmak
- İndeks tablosu oluşturmak
- Kullanıcıdan anahtar kelimeleri almak
- Belirli bir modele göre anahtar kelimeleri dokümanlarla eşleştirmek
- Dokümanları benzerliklerine göre sıralamak

1.3 Metin Madenciliği ve Bilgi Erişimi

Hocaya göre bu sayılanların içinde metin madenciliği (text mining) vardır. HTML dosyalarını web'den indirip üzerlerinde bir madencilik işlemi yapıyoruz. Bundan kasıt şudur: Dokümanların içinden önemli olan, dokümanı temsil edebilecek kelimeleri çıkarıyor ve bunlardan bir indeks tablosu oluşturuyoruz. Bu süreçte metin madenciliği yaparken konuştuğumuz yöntemler, bilgi erişimi (information retrieval) yöntemleridir.

Hoca ayrıca bu bölümün ve hatırladığı kadarıyla 8. bölüm olan tarama (crawling) bölümünün, web arama motorlarının arka planda nasıl çalıştığına ve hangi algoritmaları kullandığına odaklandığını söyledi.

2. Bilgi Erişimi ile Veri Tabanı Sorgusu Arasındaki Fark

Bilgi erişiminde, herhangi bir yapıya sahip olmayan dokümanlar üzerinde aradığımız bilgiyi bulmaya çalışırız. Veri tabanında da arama yaparız, ama fark hem saklamada hem sorguda ortaya çıkar.

- **Veri tabanı:** Veriler tablolarda tutulur. Tabloların yapısı, kolonları ve anahtarları vardır. Tablolar bu anahtarlarla birbirine bağlanarak aranır. Sorgular da SQL tabanlıdır. Yani hem arama hem saklama yapılıdır (structured).
- **Bilgi erişimi:** Dokümanların içindeki metinlerin bir yapısı yoktur. Arama yaparken de bir yapı yoktur.

Bilgi erişiminde arama için birçok alternatif vardır. Tek bir kelimeyle ya da kelime grubuyla arama yapılabilir. Kelimeler iki tırnak içine alınıp ifade (phrase) haline getirilebilir. Bir dokümanın tamamı ya da bir resmin tamamı kullanılarak da arama yapılabilir.

> **Ders sonrası not:** Hoca veri tabanı tablolarındaki anahtarlar için "private key" ve "public key" dedi; büyük olasılıkla birincil anahtar (primary key) ve yabancı anahtar (foreign key) kastedildi.

3. Arama Motorları ve Sonuçtan Bilgeliğe

3.1 Bilgi Erişimi Araçları

Bilgi erişiminin temel araçları arama motorlarıdır (search engine). Hoca bunları iki gruba ayırdı:

- **Genel arama motorları:** Web üzerinde arama yapan, herkese açık motorlardır. Bing, Google, Yahoo ve Yandex gibi.
- **Kendi alanımızdaki dokümanlar için araçlar:** Kurum içi ağ (intranet) üzerindeki dokümanları aramaya yarar. Bunlar ya Nutch gibi açık kaynaklı uygulamalarla ya da Google Site gibi araçlarla, yani genel arama motorlarının bir alana odaklanmış kullanımıyla çalışır.

Bu motorlara bilgi erişimi motoru (information retrieval engine) da denebilir. Aradığımız şeyi sorgularla (query) ifade ederiz. Motor da bu sorgulara karşılık gelen dokümanları bulur ve sıralı biçimde bize gösterir.

3.2 Sonuç Listesinden Bilgeliğe

Hoca, dersin ilk haftalarında gösterdiği bir slaytı yeniden açtı. Başlangıçta elimizde ham veri vardır. Web içerik madenciliğinde bu veri ham web sayfalarıdır. Bu sayfalar üzerinde bir bilgi erişimi motoru kullanılarak sorgular yapılır ve motor bir sonuç listesi getirir. Motorlar sonuçları genellikle onarlı listeler halinde gösterir. Birdenbire 50 ya da 100 sonuç göstermezler.

Devamında dört kavram birbirine bağlanır:

- **Bilgi (information):** Gördüğümüz sonuç listesidir.
- **Bilgi birikimi (knowledge):** Sonuçlardaki birinci, ikinci, üçüncü linke tıklarız. Her linkte bir şeyler okur ve öğreniriz. Listedeki bütün linkleri inceledikten sonra kafamızda oluşan resim bilgi birikimidir.
- **Bilgelik (wisdom):** Az önceki resim tek bir kullanıcının kafasındadır. Aynı sorguları 100 kişi kullanıyor ve 100 kişinin de kafasında aynı resim oluşuyorsa, oluşan bu bilgiye bilgelik denir.

4. Web Araması: Bilgi Erişiminin Özel Bir Hali

Bilgi erişimi, dokümanlar üzerinde bilgi aranması için kullanılan genel terimdir ve bir araştırma alanının adıdır. Web araması (web search) ise bilgi erişiminin web dokümanlarına odaklanmış şeklidir. Büyük ölçekli veriyle uğraştığı için daha baskın bir terim gibi görünür, ama aslında bilgi erişiminin web sayfalarına özel bir versiyonudur.

Bing, Yahoo ve Yandex birer bilgi erişimi motorudur. Bir kurum içi ağda çalışan yerel uygulama da bir bilgi erişimi motorudur. Web araması terimi ise bu işi büyük ölçekli ve dağıtık veri üzerinde yapan motorlar için kullanılır.

5. Web Arama Motorlarının Çözmesi Gereken Problemler

Hoca şu soruyu sordu: Normal bir bilgi erişimi motorunda görmediğimiz ama web arama motorunda karşılaştığımız problemler nelerdir?

5.1 Köprü Bağlantıları

Elinizdeki bir doküman setinde köprü bağlantısı olmayabilir. Örneğin 100 PDF dokümanınız vardır ve üzerine bir bilgi erişimi motoru koyup arama yaptırabilirsiniz. Web dünyasında ise HTML dokümanlarıyla çalışırsınız. Bu dokümanlar köprü bağlantılarıyla birbirine yumuşak bağlar (softlink) kurar. Dolayısıyla bu bağlantılar kullanılarak aramanın kalitesi artırılabilir.

Hocaya göre bugünkü arama motorlarının bu kadar başarılı sonuçlar vermesinin sebebi, sayfalar arasındaki köprü bağlantılarının avantajını kullanmalarıdır. Bunun için birçok algoritma vardır. En bilineni PageRank algoritmasıdır.

5.2 Verinin Dağıtık Olması

Web verisi merkezi bir ortamda durmaz. Her veri için ayrı bir sunucuya HTTP isteği göndermeniz ve bu isteğe karşılık gelen HTML'i almanız gerekir. Her sayfa başka bir alan adındaki sayfaya link verebilir. Bu linkleri takip ederek sürekli farklı HTTP sunucularına ulaşmanız gerekir. Bu yüzden veriyi toplamak, yani tarama işlemi, kendi başına zorlu bir iştir.

5.3 Hacim

Çok sayıda HTML sayfasına ulaşmak gerektiği için veri çok büyüktür. Bu sayfaları merkezi bir ortamda toplayıp üzerinde sorgu yapılabilecek şekilde saklamak da başlı başına büyük bir problemdir.

5.4 Güncellik

Bir sayfayı bugün tarayıp saklamanız yetmez. Veriyi bir kere alıp bırakamazsınız. Tarayıcınızın (crawler) sürekli gidip veriyi tekrar alması ve mevcut veriyi güncellemesi gerekir. Daha önce taranmış veriyi güncel tutmak da ayrı bir problemdir. Hoca, bu sayılanların hepsinin arama motorlarının her gün yaptığı tarama işleminde çözmesi gereken problemler olduğunu söyledi.

5.5 Çok Sayıda Eş Zamanlı Kullanıcı

Arama motorları yalnızca veriyi toplayıp saklamaz ve güncel tutmaz. Çok sayıda kullanıcıya eş zamanlı arama hizmeti de vermek zorundadır. Google ya da Bing'e bir anahtar kelime girince sonuçların gelmesi, arama motorunun yalnızca bize bakan tarafıdır. Biz yalnızca indekslenmiş veri üzerinde arama yaparız. O indeksi oluşturana kadar arka planda çok fazla iş yapılır.

5.6 Web Spam

Spam denince akla önce e-posta hesabımıza gelen istenmeyen iletiler gelir, ama web ortamında da spam vardır. Hoca iki biçim anlattı:

- **Görünmeyen anahtar kelimeler:** Bir sayfanın meta etiketlerine (meta tag) ya da HTML'in içine, sayfada işlenmeyecek (render edilmeyecek) ve kullanıcıya gösterilmeyecek şekilde çok sayıda anahtar kelime eklenebilir. Böylece sayfanız aslında bahsetmediği konularda da arama sonuçlarına çıkabilir.
- **Sanal link ağı:** Birbirine köprü bağlantısı veren çok sayıda sayfadan oluşan sanal bir ağ kurulup arama motoruna indekslettirilmeye çalışılabilir. Bütün sayfalar size ait olduğu ve link yapısını siz kurduğunuz için kendi sayfanıza çok sayıda link verebilirsiniz. Böylece arama motoruna, sayfanızın olması gerekenden daha önemli olduğunu düşündürebilirsiniz.

5.7 Arama Motoru Optimizasyonu

Arama motoru optimizasyonu (Search Engine Optimization, SEO) yapan firmalar, hocaya göre son anlatılan şeyi yasal bir şekilde yapmaya çalışır. Bu firmaların web'in link yapısı içinde popüler konumda olan sayfaları vardır. Aldığınız üyeliğin derecesine göre o sayfalardan sizin sayfanıza link verilmesini sağlayabilirler. Ayrıca sayfanızın yapısını optimize eder ve uygun anahtar kelimelerle sayfanızı indeksletmeye çalışırlar. Böylece sayfanız arama sonuçlarında üst sıralarda çıkar.

Hoca bu bölümü, sayılan işlerin hepsini web verisi üzerinde çalışan bilgi erişimi motorlarının yapmak zorunda olduğunu söyleyerek bağladı.

6. Bilgi Erişimi Modelinin Bileşenleri

Bilgi erişimi motorları belirli modellere dayanarak çalışır. Bir bilgi erişimi modeli, motorun işlevlerini yerine getirebilmesi için gereken şeyleri tanımlar. Hoca bunları dört başlıkta topladı:

- Sorgu dili (query language)
- Sorguların iç gösterimi (internal representation of queries)
- Dokümanların iç gösterimi
- Sıralama fonksiyonu (ranking function)

6.1 Sorgu Dili

Sorguları anahtar kelimeler, kelime grupları ya da iki tırnak arasına alınmış ifadeler şeklinde gireriz. Motor bu girdiyi olduğu gibi kullanmaz. Dokümanlarla karşılaştırabilmek için kendi içinde bir formata dönüştürür. Bu yüzden motorun kendine ait bir sorgu dili olması gerekir.

6.2 Sorgu ve Dokümanın İç Gösterimi

Bir tarafta dokümanlar, diğer tarafta sorgular vardır. Sorguları kullanıcı girer. Dokümanları ise bir ön işlemeden (pre-processing) geçirerek indekslenmiş dokümanlar haline getiririz. Bunun için dokümanları bir tanımlama yöntemiyle ifade etmemiz gerekir.

Hem sorgular hem dokümanlar bir tanımlama yöntemine göre tanımlanır. Yani ikisi de vektörlere dönüştürülür. Doküman ile sorgu ancak bu sayede birbiriyle karşılaştırılabilir hale gelir.

6.3 Sıralama Fonksiyonu

Sorguyu da dokümanı da vektör olarak tanımladıktan sonra ikisini bir karşılaştırma fonksiyonuyla karşılaştırır ve dokümanları sıralarız. Yani her dokümana bir benzerlik puanı atamaya çalışırız. Doküman ile sorgu arasındaki benzerlik için kullanılan ölçütler vardır. Bunlar aşağıda 12. bölümde ele alınıyor.

7. Sistem Mimarisi ve Çalıştırılabilir Sorgu

Hoca dört bileşeni bir mimari slaytı üzerinde birleştirdi. Dokümanlar tarama işlemiyle bir noktada toplanmıştır. Buna doküman koleksiyonu (document collection) denir. Slayttaki akış şöyledir:

```
Doküman koleksiyonu --> İndeksleyici --> İndeks
Kullanıcı --> Kullanıcı arayüzü --> Sorgu işlemleri --> Çalıştırılabilir sorgu
Çalıştırılabilir sorgu + İndeks --> Erişim sistemi (sıralama fonksiyonu) --
> Sıralı dokümanlar --> Kullanıcı
Kullanıcı geri bildirimi --> Sorgu işlemleri
```

Doküman koleksiyonu üzerinde bir indeksleyiciyle indeks oluşturulur ve arama bu indeks üzerinde yapılır. Kullanıcı, sağlanan arayüzle sorgularını oluşturur. Motor bu sorgular üzerinde bazı işlemler yaparak onları çalıştırılabilir sorguya (executable query) dönüştürür.

7.1 Çalıştırılabilir Sorgu Örneği

Kullanıcı “web data mining” diye üç kelimelik bir grup vermiş olsun. Motor bu kelimelerin arasına iki farklı operatör koyabilir:

- **AND ile:** “web AND data AND mining” sorgusu, içinde web, data ve mining kelimelerinin üçü de olan dokümanları bulur.
- **OR ile:** “web OR data OR mining” sorgusunda dokümanda yalnız web, yalnız data, yalnız mining, ikisi ya da üçü olabilir. Bu dokümanların hepsi bulunur.

Çalıştırılabilir sorguyu indeksle karşılaştıran ve indeksteki dokümanları sıralı biçimde veren kısım, yani sıralama fonksiyonu, erişim sistemi (retrieval system) modülünün içindedir.

7.2 Kullanıcı Geri Bildirimi

Slayttaki geri bildirim oku iki yere karşılık gelir. Birincisi, çalıştırılabilir sorgu çalıştırılmadan önce kullanıcıya gösterilip ondan geri bildirim istenebilir. İkincisi, sonuçlar gösterildikten sonra kullanıcı bu sonuçlar üzerinde tekrarlı (iteratif) ikinci bir arama yapmak isterse çalıştırılabilir sorguyu güncelleyebilir.

8. Sorgu Dili Örnekleri: Dijital Kütüphane Arayüzleri

Hoca, sorgu dilinin kullanıcı arayüzüyle nasıl tanımlandığını göstermek için dersten çıkıp dijital kütüphanelerin arama ekranlarını açtı. ACM’in arama motorunda şunlar görülür:

- Aramanın hangi alanda yapılacağı seçilebilir.
- Tam eşleşme (exact match) istenip istenmediği belirtilebilir.
- Birden fazla alan kullanılarak arama yapılabilir.

Birden fazla ölçüt varsa bunlar arasında mantıksal işlemler (logical operation) kullanılabilir. Örneğin ilk ikisi arasına AND, üçüncüsüyle arasına OR konabilir. Hoca bunu, mantıksal operatör seçenekleri bulunan başka bir arama motorunda gösterdi.

Hocanın vurgulamak istediği şudur: Bu seçeneklerin hepsi, kullanıcı arayüzüyle son kullanıcıya verilen sorgu dilinin parçasıdır. Kullanıcı bunları kullanarak sorgusunu oluşturur. Arama başlayınca motor anahtar kelimeleri, kullanıcının istediği mantıksal işlemleri ve her kelimenin hangi alanda aranacağı bilgisini birleştirip çalıştırılabilir sorguya dönüştürür. İstenirse bu sorgu çalıştırılmadan önce kullanıcıya gösterilebilir ya da doğrudan arama yapılabilir.

9. Sorgu Tipleri

Bir bilgi erişimi motoru farklı tiplerde sorgular sağlayabilir. Hoca altı tip saydı.

9.1 Anahtar Kelime Sorguları

Anahtar kelime sorguları (keyword queries), Google, Bing, Yahoo ve Yandex gibi genel arama motorlarının sağladığı temel özelliktir. Bir anahtar kelime ya da kelime grubu girilerek arama yapılır.

9.2 Boolean Sorgular

Hem genel motorlarda hem de dijital kütüphanelerin motorlarında Boolean sorgular oluşturulabilir. Sorgunun parçaları AND, OR ve NOT gibi operatörlerle birleştirilir. Bu durumda motor yalnızca sorguya doğrudan karşılık gelen, yani sorguyu doğru (true) yapan dokümanları bulur.

Örnek olarak “web AND data NOT mining” sorgusu şu anlama gelir: Dokümanda web ve data mutlaka bulunacak, mining ise kesinlikle bulunmayacak. Bulunan dokümanların hepsi bu koşul için doğru olmak zorundadır.

9.3 İfade Sorguları

Kelime grubu iki tırnak içine alınarak aranırsa bu bir ifade sorgusudur (phrase query). Bu sorgu motora şunu söyler: Kelimeleri aynı sırada ve aynı şekilde içeren dokümanlarla ilgileniyorum. Tırnak içinde “web data mining” yazılırsa bu üçlüyü olduğu gibi içeren dokümanlar aranır.

9.4 Yakınlık Sorguları

Yakınlık sorgusunda (proximity query) web, data ve mining kelimelerinin dokümanda birbirine yakın olması istenir. Kelimelerin peş peşe gelmesi şart değildir. Hocanın gösterdiği arama ekranında bu özellik yoktu, ama başka özel amaçlı arama motorlarında vardır.

Hoca şu örneği verdi: Motor iki doküman bulmuş olsun. Birincisinde web kelimesi dokümanın en başında, data ortasında, mining en altındadır. İkincisinde kelimeler peş peşe değildir ama aynı paragraf içinde, yani birbirine çok yakındır. Bu durumda motorun ikinci dokümanı daha ön sıralarda göstermesi gerekir.

Yakınlık sorgusunun ifade sorgusundan farkı budur: Kelimelerin arasında başka kelimeler olabilir, yeter ki yakın konumlarda olsunlar.

9.5 Tam Doküman Sorguları

Tam doküman sorgusunda (full document query) arama motoruna bir dokümanın tamamı verilir. Motor buna benzer dokümanları bulmaya çalışır. Hoca örnek olarak Google’ın görsel aramasını ve PDF aranabilmesini gösterdi.

9.6 Doğal Dil Soruları

Son tipte soru doğal dille sorulur (natural language question). Bing ve Google gibi motorlar bunu desteklemeye çalışır. Hocanın örneği “dolar şu anda kaç TL” sorusudur. Motor girilen

kelimelere karşılık gelen sonuçları yine listeler. Ayrıca soruya doğrudan karşılık gelen yanıtı en tepede gösterir. Böylece doğal dil işleme soruları da sorgu tiplerinden biri haline gelmiştir.

10. Boolean Bilgi Erişimi Modeli

Sorguların ve dokümanların iç gösterimi için farklı bilgi erişimi modelleri kullanılır. Hoca üç tanesini saydı: Boolean model, vektör uzayı modeli (vector space model) ve istatistiksel dil modeli (statistical language model). Bugün en yaygın kullanılan vektör uzayı modelidir. Boolean model ve istatistiksel dil modeli de kullanılır. Derste ilk ikisi anlatıldı.

Slayttaki ortak gösterimde bir doküman, V tane kelimenin her biri için bir ağırlık değeri içeren bir vektör olarak yazılır:

$$d_j = (w_{1j}, w_{2j}, ..., w_{|V|j})$$

Tanımlama yöntemini belirleyen şey, bu ağırlığın ne olacağıdır.

10.1 Ayırt Edici Kelimeler ve Kelime Dağarcığı

Doküman, içindeki ayırt edici kelimeler (distinctive) kullanılarak vektöre dönüştürülür. Dokümandaki her kelime ayırt edici değildir. Bağlaçlar ve bazı dillerdeki artikeller gibi belirli bir anlam taşımayan, yalnız doğal dilin gereği olarak kullanılan ve çok sayıda cümlede ya da dokümanda tekrar edebilen kelimeler vardır. Bunlar kullanılmaz. Yalnızca anlamı olan ve dokümanı tanımlarken ayırt edici sayılabilecek kelimeler alınır.

> **Ek bilgi (derste anlatılmadı):** Bu tür işlevsel ve çok sık geçen kelimelere metin madenciliğinde durak kelime (stop word) denir. Çoğu sistem bunları indekslemeden önce bir liste yardımıyla çıkarır.

Bu kelimelerin oluşturduğu kümeye kelime dağarcığı (vocabulary) denir. Hoca burada önemli bir ayrım yaptı. Türkçenin ya da İngilizcenin bir kelime hacmi vardır. Bununla o dili konuşmak için toplamda kullanılabilecek kelime sayısı kastedilir. Vocabulary ise bir dilin tüm kelimeleri değildir. Örneğin 100 dokümanlık bir set düşünün. Bu setteki anlamlı kelimelerin hepsini bir araya topladığınızda o set için bir vocabulary elde edersiniz. Yani elimizdeki dokümanların içerdiği kelimelerden söz ediyoruz.

Vocabulary'de V tane kelime varsa setteki her doküman V elemanlı bir vektörle tanımlanır. Bu kelimelerin bazıları belirli bir dokümanda bulunur, bazıları bulunmaz. Çünkü kelimeler tek bir dokümandan değil, bütün doküman setinden çıkarılmıştır.

10.2 Dokümanın ve Sorgunun Gösterimi

Boolean modelde kural basittir: Kelime dokümanda varsa değeri 1, yoksa 0'dır. Modelin Boolean adını almasının sebebi de budur. Başka bir ağırlık kullanılmaz.

> **Ders sonrası not:** Hoca 0 değeri için "kelime bu doküman setinde yoksa" dedi; doğrusu "kelime bu dokümanda yoksa". Vocabulary setten çıkarıldığı için kelime setin içinde zaten vardır. Hoca aynı konuda bir yerde "bu dokümanın vocabulary'si" dese de, hemen sonrasında vocabulary'nin doküman setine ait olduğunu açıkladı.

Bir bilgi erişimi modelinde dokümanın olduğu kadar sorgunun da gösterimi tanımlanmalıdır. Sorguyu tanımlamak için kullanıcıya, az önceki arayüzlerde görüldüğü gibi, bazı operatörler verilir. Kullanıcı bunlarla sorgusunu kurar. Hocanın slayttaki örneği "data AND mining AND NOT text" sorgusudur. Dokümanda data ve mining mutlaka olacak, text olmayacaktır.

10.3 Tam Eşleşme

Boolean modelde sonuçlarda yalnız sorguyu doğru yapan dokümanlar gösterilir. Yani yalnız tam eşleşen (exact match) dokümanlar döner. Çalıştırılabilir sorgu da doküman gibi 1'den V'ye kadarki kelime uzayında ifade edilebilir. Böylece sorgu ile doküman karşılaştırılıp tam eşleşenler bulunur.

11. Vektör Uzayı Modeli ve TF-IDF

Vektör uzayı modeli, günümüzdeki genel arama motorlarında yaygın olarak kullanılan bilgi erişimi modelidir. Burada da dokümanlardan ayırt edici kelimeler çıkarılır. Bu kelimelerin oluşturduğu kelime uzayı kullanılarak her doküman bir vektör olarak tanımlanır.

Fark ağırlıktadır. Boolean modelde kelime varsa 1, yoksa 0 yazılıyordu. Vektör uzayı modelinde ise kelime varsa onu tanımlamak için TF-IDF adlı bir ağırlıklandırma yöntemi kullanılır. TF-IDF, bir kelimenin bir doküman içindeki önemini belirten bir ölçüttür. İki parçadan oluşur.

11.1 Terim Frekansı

Örneğin doküman setinde 100 doküman olsun ve bunlardan birini seçelim. Bu dokümandaki bir kelimenin TF-IDF skorunu hesaplamak istiyoruz. Önce terim frekansına (term frequency, TF) bakılır. Pay, kelimenin bu dokümanda kaç kez tekrar ettiğidir. Yalnız payı alsaydık yalnızca kelimenin frekansını almış olurduk. Bu yüzden normalize ederiz: Payda, aynı dokümanda en yüksek frekansa sahip kelimenin frekansıdır.

$$tf_{ij} = \frac{f_{ij}}{\max\{f_{1j}, f_{2j}, \dots, f_{|V|j}\}}$$

Burada f_{ij}, i kelimesinin j dokümanında kaç kez geçtiğidir. Bu oran, kelimenin o doküman içindeki önemini gösterir.

> **Ders sonrası not:** Hoca TF paydasını anlatırken önce "doküman setinde bulunan bu kelimenin diğer dokümanlar arasındaki maksimumu" gibi bir ifade kullandı, sonra düzeltti; doğrusu, aynı doküman içinde en sık geçen kelimenin frekansıdır. Ayrıca hoca sözlü anlatımda i'yi doküman, j'yi kelime için kullandı. Yukarıdaki formüllerde ise i kelime, j dokümandır.

11.2 Ters Doküman Frekansı

İkinci parça ters doküman frekansıdır (inverse document frequency, IDF). Burada şuna bakılır: 100 dokümanlık uzayda bu kelime kaç dokümanda geçiyor?

Hoca sezgiyi şöyle kurdu: Bir kelime 100 dokümanın tamamında geçiyorsa onu aradığımızda 100 dokümanın hepsi gelir. Yani ayırt edici özelliği düşüktür. Kelime az sayıda dokümanda geçiyorsa onu girdiğimizde az sayıda sonuç gelir. Yani ayırt edici değeri yüksektir.

$$idf_i = \log \frac{N}{df_i}$$

Burada N toplam doküman sayısı, df_i ise i kelimesinin geçtiği doküman sayısıdır. Kelimenin geçtiği doküman sayısı, yani payda büyüdükçe IDF skoru düşer. Payda küçüldükçe skor yükselir.

Hocanın 100 doküman örneği formüle konursa (ara adım eklendi): $N = 100$, $df_i = 100$ için $idf_i = \log(100/100) = \log 1 = 0$. Logaritmanın tabanı ne olursa olsun sonuç 0'dır. Yani hocanın "ayırt edici özelliği düşük" dediği kelimenin IDF değeri bu formülle tam olarak sıfır çıkar.

> **Ders sonrası not:** Hoca "bu i'nci doküman için IDF'i hesaplıyorum" dedi; doğrusu IDF dokümana değil kelimeye ait bir değerdir ve doküman setinin tamamından hesaplanır. Hocanın devamdaki anlatımı da bu şekildedir.

11.3 TF-IDF Ağırlığı

İki değer çarpılır:

$$w_{ij} = tf_{ij} \times idf_i$$

TF, kelimenin bir doküman üzerindeki önemini belirtir. IDF ise kelimenin doküman seti içindeki önemini belirtir. Bir kelime dokümanda çok tekrar ediyorsa, özellikle de diğer kelimelerden daha fazla tekrar ediyorsa TF büyür. Kelime sette az sayıda dokümanda, yani nadir görünüyorsa IDF büyür. Dolayısıyla ağırlığı da büyür. Bu skorla bir kelimenin hem bulunduğu dokümandaki hem de doküman uzayındaki önem değeri hesaplanmış olur.

11.4 Vektörü Oluşturmak

Boolean modelde dokümanı, kelime uzayındaki her kelime için bir eleman içeren vektörle ifade ediyor ve bu elemanlara 1 ya da 0 yazıyorduk. TF-IDF kullanınca bu elemanlara kelimelerin TF-IDF skorları yazılır. Kimi zaman yalnız TF skoru da konabilir. Bu işlem hem doküman hem de sorgu için yapılır.

12. Sıralama Fonksiyonu: Kosinüs ve Okapi Benzerliği

Sorgu ve doküman için iki vektör oluşturulduktan sonra bunlar sıralama fonksiyonuyla karşılaştırılır.

12.1 Kosinüs Benzerliği

Hocanın verdiği örnek kosinüs benzerliğidir (cosine similarity). İki vektör arasındaki benzerliği bir gerçel sayı olarak verir. İki vektörün iç çarpımı (dot product) alınır ve vektörlerin uzunluklarının çarpımına bölünür:

$$cosine(d_j, q) = \frac{\langle d_j \cdot q \rangle}{\|d_j\| \times \|q\|} = \frac{\sum_{i=1}^{|V|} w_{ij} \times w_{iq}}{\sqrt{\sum_{i=1}^{|V|} w_{ij}^2} \times \sqrt{\sum_{i=1}^{|V|} w_{iq}^2}}$$

> **Hocanın vurgusu:** İki vektörün iç çarpımının nasıl alındığına ve uzunluklarının nasıl bulunduğuna aşina olmanız gerekir. “Bununla karşılaşabilirsiniz.”
>
> **Ders sonrası not:** Hoca iki yerde kosinüs için “uzaklık” ve “uzaklık metriği” dedi; doğrusu benzerlik (similarity) ölçüsüdür. Değer büyüdükçe iki vektör birbirine daha yakın sayılır. Hocanın kendi anlatımı da en yüksek değerden en düşüğe doğru sıralamadır.
>
> **Ek bilgi (derste anlatılmadı):** Adındaki kosinüs, iki vektör arasındaki açının kosinüsüdür. Vektör uzunluklarına bölündüğü için ölçüm vektörlerin büyüklüğünden çok yönlerine bakar. Ağırlıklar negatif olmadığında değer 0 ile 1 arasındadır. Hiç ortak kelime yoksa 0, iki vektör aynı yönde ise 1 çıkar.

12.2 Oyuncak Örnek

Hoca slaytta bir oyuncak örnek gösterdi. Doküman uzayında 3 kelime ve 9 dokümanlık bir set vardır. Her doküman üçlü bir vektörle ifade edilir: Kelime varsa 1, yoksa 0. Yani örnekteki ağırlıklandırma Boolean modeldir. Dokümanlar $A_1, A_2, \ldots$ diye adlandırılmıştır. Sorgu hardware ve software kelimelerini içerir ve hangi dokümanların getirileceği sorulmaktadır.

Üç farklı sıralama yaklaşımı karşılaştırıldı:

- **Boolean eşleştirme, AND:** Hardware ve software’in ikisi de olmalıdır. Bu koşulu sağlayanlar A_4 ve A_7’dir. Yalnız bu ikisi getirilir.
- **Boolean eşleştirme, OR:** İki kelimeden birinin 1 olması yeter. İkisinin de 0 olduğu tek doküman A_3’tür. Bu yüzden A_3 dışındaki bütün dokümanlar getirilir.
- **Kosinüs benzerliği:** Her doküman için farklı bir değer çıkar. Dokümanlar en yüksek değerden en düşüğe doğru sıralanarak kullanıcıya gösterilir.

Burada hocanın dikkat çektiği önemli bir nokta var. Örnekte hem sorgu hem dokümanlar 1 ve 0’lı ağırlıklarla tanımlıdır, yani gösterim Boolean modeldedir. Fakat sıralama fonksiyonu olarak tam eşleşme yerine bir benzerlik fonksiyonu da seçilebilir. Gösterim ile sıralama fonksiyonu ayrı seçimlerdir.

Bir fark daha vardır: Boolean eşleştirme yaparken üçüncü elemanın 0 mı 1 mi olduğunu söylemek gerekmez. Sorgu yalnızca ilk iki elemana bakılarak çalıştırılabilir. Kosinüs benzerliğinde ise vektör karşılaştırması yapıldığı için sorgu vektörünün tamamı belirtilmelidir. Kullanıcı üçüncü kelime hakkında bilgi vermediyse demek ki onu istemiyordur diye düşünülür ve 0 yazılır. Yani sorgu vektörü $q = (1, 1, 0)$ olur.

Slayttaki değerler altyazıdan okunamadığı için aşağıda yalnız bu sorgu vektöründen doğrudan çıkan genel ifade verilmiştir (ara adım eklendi). Sorgunun uzunluğu $\|q\| = \sqrt{1^2 + 1^2 + 0^2} = \sqrt{2}$'dir. İkili bir doküman vektöründe, sorgudaki iki kelimeden k tanesi geçiyorsa iç çarpım k olur. Dokümanda toplam m tane 1 varsa uzunluğu $\sqrt{m}$'dir. Dolayısıyla:

$$cosine(d, q) = \frac{k}{\sqrt{2} \times \sqrt{m}}$$

Buna göre iki kelimeyi de içeren A_4 ve A_7 için $k = 2$, iki kelimesi de 0 olan A_3 için $k = 0$'dır. A_3'ün iç çarpımı 0 olduğundan, vektörü bütünüyle sıfır değilse kosinüs benzerliği de 0 çıkar. A_4 ve A_7'nin değerleri ise üçüncü elemanlarına bağlıdır.

12.3 Okapi Benzerliği

Kosinüs benzerliği tek seçenek değildir. Bir dokümanın diğerine ne kadar yakın olduğunu ifade etmek için farklı yöntemler vardır. Bunlardan biri literatürde önerilmiş Okapi benzerliğidir. Hesabı biraz daha karmaşıktır, ama deneysel çalışmalar oldukça başarılı sonuçlar verdiğini göstermiştir. Hoca formülüne girmedi.

> **Ek bilgi (derste anlatılmadı):** Okapi ailesinin bugün en bilinen biçimi büyük olasılıkla BM25'tir. Ana fikri, TF-IDF'e iki düzeltme eklemektir. Birincisi, bir kelimenin tekrar sayısı arttıkça katkısı doğrusal değil, gittikçe azalarak büyür. İkincisi, uzun dokümanlar doküman uzunluğuna göre normalize edilir.

13. İlgililik Geri Bildirimi

Hoca arama sonuçları gösterildikten sonraki aşamaya geçti. Kullanıcı sonuç listesinde bazı linklere tıklayabilir. Bununla sistemin gösterdiği sonuçlar hakkında bir geri bildirim vermiş olur. Örneğin 10 sonuçtan 3'üne tıklanıyorsa, bu 3 sonucun aranan şeyle daha ilgili olduğu düşünülebilir.

Hoca yöntemi şöyle anlattı: İlgili (relevant) sayılan dokümanların içinde çok sayıda tekrar eden kelimeler çıkarılır. Aynı sorgu bir daha girildiğinde bu sorgu yeni kelimelerle genişletilir (query expansion). Bu tür yöntemlere ilgililik geri bildirimi (relevance feedback) denir. Amaç, sorgunun başarısını artırmak ve kullanıcının aradığı sayfalara daha hızlı ulaşmasını sağlamaktır.

Soru: Kullanıcıya 10 doküman gösterildi diyelim. İlk linke tıkladı, aradığını bulamadı, geri geldi ve ikinci linke tıkladı. Aradığı doğru belge ikincisiydi. Bu durumda birinci linki ilgili saymak yanlış olmaz mı?

Cevap: Haklısın. Tıklanan linki ilgili saymayı söylememem gerekirdi. Şöyle bir arama motoru tasarladığımı düşün: Kullanıcı metin kutusuna anahtar kelimelerini girer. Ona 10 sonuç gösterilir ve her sonucun yanında bir onay kutusu (checkbox) bulunur. Kullanıcıdan sayfalara girip beğendiklerini işaretlemesi ve tekrar arama butonuna basması istenir. İşaretlenen sayfalardan elde edilen ek kelimeler, kullanıcının ilk girdiği kelimelere eklenir. Oluşan yeni kelime grubuyla arama yapılır ve bir sonraki turda sonuçların kalitesi çok daha yüksek olur.

Hoca bu cevabın devamında açık ve örtük geri bildirimi ayırdı:

- **Açık (explicit) geri bildirim:** Onay kutularıyla kullanıcıdan ilgili sonuçları işaretlemesi istenir. İlgililik geri bildirimi tabanlı arama bu tür motorlarda yapılabilir.
- **Örtük (implicit) çıkarım:** Genel arama motorlarında onay kutusu özelliği yoktur. Bu motorlar yalnızca kullanıcının tıklamalarından çıkarım yapmaya çalışır.

Ders sonrası not: Hoca yöntemi ilk anlatışında tıklanan linkleri ilgili doküman saydı, öğrencinin itirazı üzerine düzeltti; doğrusu, tıklanmış olmak tek başına ilgili olduğunu göstermez. İlgililik bilgisi, yukarıdaki gibi kullanıcının açıkça işaretlemesiyle alınır.

14. Dönem Projesi Sunumları

Konu anlatımı burada durdu. Hoca, metin ön işleme konusuna geçmeden önce öğrencilerin proje önerilerini ekran paylaşımıyla tek tek ele aldı. Altı öneri konuşuldu. Hoca her öneride neyi net görmek istediğini söylediği için bu bölüm, bu tür bir web madenciliği projesinin nasıl kurulması gerektiğine dair de bilgi içerir.

14.1 1. Sunum: Akademik Yayın Arama Motoru

Konu: Bir üniversitedeki araştırmacıların yayınları içinde arama.

Amaç: Google Scholar'da bir üniversitenin araştırmacılarının yayınları içinde konu bazlı arama yapılamaması ve profillere tek tek bakmak gerekmesi. Öğrenci, belirli bir konuda hangi araştırmacıların çalıştığını bulabilmek istiyor.

Önerilen yöntem: Seçilen üniversitedeki araştırmacıların verilerini Google Scholar'dan alıp derste anlatılan kavramlarla bunlar üzerinde sonuç döndüren bir uygulama geliştirmek.

Hocanın geri bildirimi: Hoca, Google Scholar'ın tek bir kaynak olduğunu söyledi. Yayınlar Scopus, Web of Science, DBLP gibi farklı veri tabanlarında da paylaşılır. Bunun üzerine öneriyi hibrit bir akademik yayın arama motoruna dönüştürdü:

- Bir açılır menüden üniversite seçilir, örneğin Yıldız Teknik Üniversitesi ya da Boğaziçi Üniversitesi.
- Bir metin kutusuna anahtar kelimeler girilir, örneğin "web data mining".
- Önce bir kaynaktan o üniversitede yayını olan araştırmacılar öğrenilir. Ardından yayınlar yalnız bir yerden değil, en az üç farklı ortamdan toplanır.
- Sistem o üniversite için kurulabilir, ama üniversite değiştirildiğinde kodda çok fazla değişiklik gerekmeyecek şekilde yapılandırılabilir olmalıdır.
- Aynı yayının tekrarları (duplication) kaldırılır. Sonuçlar tek satırda gösterilir ve yanına bulunduğu kaynakların ikonları ile linkleri konur. Yayın üç veri tabanında varsa tek kayıt ve üç ikon görülür.
- Sonuçlar sıralı olarak gösterilir, örneğin ilk 10 ya da ilk 20.

Hoca bu yapıyı göstermek için bir fiyat karşılaştırma sayfası açtı. Böyle bir sayfa aynı ürünü farklı firmalardan getirir. Burada da farklı kaynaklardan yayın getirilecektir.

Öğrenci, projenin sandığından çok büyüdüğünü söyledi. Hoca, tek ortamdan tarama istemediğini açıkladı. Genel kuralı da burada koydu: Bir web sayfasından veri toplayan bir proje en az 3 web sayfasından veri toplamalıdır. Kaynağın RESTful servisi olabilir ya da kendi ayrıştırıcınızı yazarak veriyi çekebilirsiniz. Tarayıcı tabanlı bir işte beklenen, birden fazla kaynaktan veri çekip birleştirmek ve hibrit olarak göstermektir.

Hoca işi öğrenci için şöyle sadeleştirdi: Veriyi tarayıp bir veri tabanı oluşturmak ve üzerinde arama yaptırmak zorunda değilsin. Sorguları bu sayfalara gönderirsin. Sayfalar sonuç üretir. Sen bu sonuç kümelerini (result set) alır, birleştirir ve gösterirsin. Yani arka plandaki arama motorlarının önüne bir hibrit arama motoru koyuyorsun. Yeni bir anahtar kelime girildiğinde, örneğin "sinyal işleme", sorgu yine bu kaynaklara gider.

Sorguların üniversiteyle sınırlı kalması gerekir. Bunun için sorgu kurulurken kurum (affiliation) bilgisi koşul olarak verilir. Hoca IEEE ekranında yazar kurumu (author affiliation) filtresini gösterdi ve her veri tabanının kurum bilgisini alabildiğini söyledi.

Soru: Google Scholar kurum bilgisi vermeye izin vermiyor. Yazar adı verilebiliyor, ama diyelim ki 9000 akademisyen var. Bu durumda 9000 dinamik sorgu göndermek gerekir ve Google bu noktada blokluyor. Ne yapılmalı?

Cevap: O zaman IEEE Xplore, ACM, DBLP ve Scopus'a geçilir. En az 3 veri tabanından sonuç gelmesi yeterlidir.

Sonuç: Öneri, “akademik yayın arama motoru” odağıyla güncellenecektir. Hangi üç dijital veri tabanının kullanılacağı raporda açıkça yazılmalıdır.

14.2 2. Sunum: Twitter’da Siber Güvenlik Tehdidi Tespiti

Konu: Açık kaynak istihbaratı (OSINT) kapsamında tweet’lerden siber tehdit yakalamak.

Amaç: Yeni bir zafiyet açığa çıkıp çıkmadığını, bir üründe güvenlik açığı bulunup bulunmadığını ya da bir saldırı planlanıp planlanmadığını tweet’lerden tespit etmek. Sonuçlar SIEM sistemleriyle entegre çalışarak alarm üretecektir.

Önerilen yöntem: Öğrencinin önerdiği akış şöyledir:

```
Tarayıcı (belirli anahtar kelimelerle Twitter sorguları, en yeni tweet'ler)
  --> Mesaj kuyruğu
  --> Sınıflandırıcı (siber olay mı, önem derecesi kaç)
  --> İndeks motoru (örneğin Elasticsearch)
  --> SIEM entegrasyonu
```

Veri tarafında NIST’in National Vulnerability Database (NVD) adıyla yayımladığı tweet’ler ve bunların CVSS skorları kullanılacaktır. Bu veri üzerinde yapılmış bir adlandırılmış varlık çalışması da vardır. Önce bir tweet’in siber olayla ilişkili olup olmadığını bulan ikili bir sınıflandırıcı (binary classifier) geliştirilecek, sonra çıkarılan bilgilerle olayın envanterdeki bir cihazla eşleşip eşleşmediğine ve önem derecesine bakılacaktır.

Hocanın geri bildirimi: Hoca şu eklemeleri istedi:

- **Veri seti:** Hangi veri seti olduğu ve linki raporda yazmalıdır. Toplam tweet sayısı, kaçının güvenlik açığı belirttiği ve kaçının normal olduğu gibi sayısal değerler bilinmelidir. Veri seti indirilebilir ve kontrol edilebilir olmalıdır.
- **Öznitelikler:** Hangi özniteliklerin (feature) çıkarılacağı raporda yazmalıdır. Hangi gömme (embedding) yönteminin kullanılacağı da belirtilmelidir, örneğin Word2Vec tabanlı yöntemler.
- **Sınıflandırıcılar:** En az 3 sınıflandırıcı kullanılmalıdır. Biri TensorFlow üzerinde Keras ile LSTM olabilir. Hocaya göre bu çok zor değildir. Diğer ikisi lojistik regresyon (logistic regression) ve rastgele orman (random forest) gibi standart makine öğrenmesi yöntemleri olabilir.
- **Değerlendirme:** Veri %70 ya da %80 eğitim, geri kalanı test olarak ayrılır. Başarı doğruluk (accuracy), kesinlik (precision), duyarlılık (recall) ve F1 ile gösterilir.
- **Gerçek zamanlı kısım:** Önce modeller oluşturulur, sonra gerçek zamanlı gelen tweet’ler etiketlenir. Ekranda tweet görünür ve hemen yanında A, B ve C modellerinin tahmini yer alır. Böylece tweet içindeki kelimelere bakılarak etiketin doğru olup olmadığı gözle de takip edilebilir.

Hoca, tweet’in vektöre nasıl çevrileceğini şöyle örnekledi: Tweet’te 5 kelime varsa her kelime bir vektöre dönüştürülür ve 5 vektör elde edilir. Bu vektörlerin ortalaması alınır ve tweet bu ortalama vektörle temsil edilir. Etiket ise tweet’in güvenlik açığıyla ilgili bilgi içerip içermediğidir. Sonuçta elde vektörlerden oluşan bir matris ile bir etiket kolonu olur. Sınıflandırıcılar bunun üzerinde eğitilir.

Soru: Projeyi ileride genişletip tweet’in etki oranını ya da önem derecesini de tahmin etmek mümkün mü?

Cevap: Önemli olan projenin önümüzdeki iki ay içinde bitirilebilecek boyutta olmasıdır. Sonrasında devam etmek öğrencinin kararıdır.

Sonuç: Rapor; üç makine öğrenmesi algoritmasını, indirilecek veri setinin linkini ve bu bilgilerin hepsini açıkça içerecek şekilde güncellenecektir.

14.3 3. Sunum: E-Ticaret Yorumlarında Fikir Madenciliği

Konu: E-ticaret sitelerindeki teknolojik ürün yorumları.

Amaç: Yorumların olumlu mu olumsuz mu olduğunu sınıflandırmak (duygu analizi, sentiment analysis). Öğrencinin motivasyonu, üreticilerin müşteriye verdiği hizmeti iyileştirmeye katkı sağlamaktır.

Önerilen yöntem: Yorum bazlı ikili etiketleme. 1 ya da 2 yıldızlı yorumlar olumsuz, 4 ya da 5 yıldızlı yorumlar olumlu sayılır. 3 yıldızlı yorumlar veri setine alınmaz.

Hocanın geri bildirimi: Hoca kitabın fikir madenciliği (opinion mining) bölümünü açtı ve projeyi özellik tabanlı fikir madenciliğine (feature-based ya da aspect-oriented opinion mining) yönlendirdi. Bütün projelerin kitaptaki bir algoritmayı kullanmasını istediğini söyledi. Örneğin 1. sunum tarama bölümüne, 2. sunum gözetimli öğrenme algoritmalarının web verisinde kullanımına odaklanıyordu.

Soru: Benim yaptığım da bir tür gözetimli etiketleme olmuyor mu?

Cevap: Hoca bir adım geri atıp beklentisini anlattı. Bir ürünle ilgili çok sayıda yorum işlenip bilgi çıkarılacaktır. Yorumun içindeki cümlelerde insanlar ürünün alt özellikleri hakkında konuşur. Slayttaki örnek dijital kameradır: resim (picture), batarya (battery), zoom, boyut (size) ve ağırlık (weight) özellikleri için yorumlar vardır. Örneğin 100 yorumun 90'ı resim özelliği için olumlu, 10'u olumsuz görüş bildirmiş olabilir. Beklenen çıktı, bu tür bir rapordur: özellik tabanlı fikir özeti (feature-based summary of opinions).

Soru: Bazı kötü yorumların ürünün özelliğiyle ilgisi yok. Örneğin kargo geç gelmiş ya da ürün pahalı bulunmuş. Bunları nasıl eleyeceğim?

Cevap: Hoca iPhone 12 örneğini verdi. 100 yorumun 80'inde resim, batarya, boyut gibi ürün özellikleri, 20'sinde ise kargo ya da sipariş arayüzü gibi konular olabilir. Algoritma, çalışma şekli gereği yalnız özellik içeren yorumları dikkate alır. Öğrenci bunu bölümdeki algoritmayı okuyarak görecektir.

Soru: Proje için bir grafik arayüz (GUI) mi isteniyor? Kod çalışınca bir görüntü zaten çıkıyor.

Cevap: Evet, GUI isteniyor. Bir açılır menüye basıldığında veri setindeki ürünler listelenir. Bir ürün seçilir, örneğin bir televizyon. Altta bir buton belirir. Butona basıldığında özellik tabanlı sonuçlar grafik olarak gösterilir.

Sonuç: Veri taranmayacak, ürün yorumlarının bulunduğu hazır bir veri seti kullanılacaktır. Raporda ürün sayısı ve ürün başına yorum sayısı gibi bilgiler verilmelidir. Hoca bunun için "10 ürün, her ürün için 500 yorum" gibi bir ifadeyi örnek gösterdi. Hoca zor bir proje istemediğini, algoritmanın örneklerinin ve veri setlerinin bulunabileceğini, daha önce başka öğrenci gruplarının bu projeyi başarıyla yaptığını söyledi.

14.4 4. Sunum: Kitap Siteleri İçin Hibrit Arama Motoru

Konu: Kitap alanında tarayıcı tabanlı bir arama motoru.

Amaç: Tarama işini ve arama motorlarının işleyişini daha iyi anlamak. Kitap adı ya da yazar adıyla arama yapılabilmesi.

Önerilen yöntem: Deneysel olarak yayımlanmış Books to Scrape sitesinden başlayıp 3 farklı kitap sitesini taramak, sonuçları arama motorunda birleştirip sıralamak.

Hocanın geri bildirimi: Hoca bunu 1. sunumdaki gibi hibrit bir arama motoru olarak yorumladı. Arka planda farklı kaynaklara istek gönderilir ve gelen veriler bir liste halinde gösterilir. İstekleri şunlardı:

- Hangi kitap sitelerinin kullanılacağı yazılmalıdır.
- Sitelerin RESTful servisi olmasına dikkat edilmelidir. Yoksa HTML sayfalarının ayrıştırılması gerekir.
- Aynı kitap A, B ve C kitapçılarında bulunduysa üç ayrı sonuç gösterilmez. Tek kayıt olur. Yanına her site için ikon konur ve kullanıcı istediğine gidebilir.
- Fiyat bilgisi de işin içinde olduğu için her sitedeki fiyat aynı satırda gösterilir. Böylece hangisinin daha ucuz olduğu görülür.

Sonuç: Rapor bu şekilde güncellenecektir.

14.5 5. Sunum: Müzik Yorumlarında Duygu Analizi

Konu: Müziklerle ilgili sosyal medya yorumlarında duygu analizi.

Amaç: Öğrencinin asıl ilgisi, Spotify verilerindeki müzik özellikleri (örneğin ritim, ölçü, ton) ile dinleyici tepkisi arasındaki ilişkidir. Spotify dinlenme sayısını veriyor, ama sosyal medyadaki görüşler de bir tepki kaynağıdır. Proje bu çalışmaya yan bir destek olarak düşünülmüştür.

Önerilen yöntem: Belirli anahtar kelimelerle ya da etiketlerle Twitter'ı taramak ve TextBlob gibi araçlarla duygu analizi yapmak. Öğrencinin gördüğü engel, Twitter'ın ücretsiz API'sinin yalnız son 7 güne izin vermesi ve bu yüzden veri biriktirmek gerekmesidir.

Hocanın geri bildirimi:

- **Hazır veri seti:** Hoca burada genel kuralını söyledi. Tarayıcı projeleri veriyi anlık toplamak zorundadır. Diğer herkes internetten indirilebilir, istatistiksel özellikleri bilinen bir veri seti üzerinde tekrarlanabilir bir çalışma yapmalıdır. Kısa süreli bir proje için veri toplamak fazladır. Toplanan veriden anlam çıkmayabilir, Twitter sayfası bloklayabilir, öngörülemeyen sorunlar yaşanabilir.
- **Tweet düzeyinde olumlu ya da olumsuz:** Tweet karakter sayısı sınırlı bir veridir ve çok fazla bilgi çıkarılamaz. Tweet'leri olumlu ya da olumsuz diye etiketlemek, hocaya göre bu ders için anlamlı bir proje değildir.
- **Daha anlamlı yol:** Fikir madenciliği yapılacaksa içeriğe girip insanların o ürün ya da müzik hakkında hangi ayrıntıları söylediğine bakan özellik tabanlı fikir madenciliği daha anlamlıdır.

Sonuç: Hoca iki yol bıraktı:

- **Fikir madenciliği:** Kitabın 11. bölümü olan fikir madenciliği bölümü okunur.
- **Sınıflandırma:** Tweet etiketi tahmin edilecekse 2. sunumdaki gibi en az 3 sınıflandırıcı kullanılır, biri LSTM olur. Tweet'lerden çıkarılan öznitelik vektörleriyle modeller oluşturulur ve yeni tweet'in etiketi tahmin edilir. Etiket olumlu ya da olumsuz olabilir.

Her iki durumda da veri setinin linki raporda olmalı, veri sayısı ve etiket durumu görülebilmeli, modellerle neyin etiketleneceği belli olmalıdır. Bir kullanıcı arayüzü de mutlaka olmalı ve modellerle tahmin bu arayüzde gösterilmelidir. Öğrenci hazır veri setlerini inceleyecektir.

> **Hocanın vurgusu:** Yalnız slaytlarda veri analizini, confusion matrix'i, accuracy ve F-skor değerlerini göstermek yetmez. "Kod yazmanızı istiyorum arkadaşlar ve bunu da demoyla göstermenizi istiyorum."

14.6 6. Sunum: YouTube Videolarının Trend Olma Tahmini

Konu: YouTube video içerik analizi ve istatistik tahmini.

Amaç: Bir video fikrinin ne kadar trend olabileceğini ve kaç kişiye ulaşabileceğini tahmin etmek.

Önerilen yöntem: Önce hazır bir veri kullanmak, vakit kalırsa Türkiye için kendi verisini çekmek. Tahmini güçlendirmek için arama motorlarından ve forumlardan gündem bilgisi de almak.

Hocanın geri bildirimi: Hoca öneriyi temel bir soruya indirdi: Video trend olacak mı, olmayacak mı? Bu bir ikili sınıflandırmadır.

Soru: Bir gün belirleyip 5 video alarak bir hafta sonrasını karşılaştırmalı mıyım, yoksa bir video için kaç kişiye ulaşabileceğini mi söylemeliyim?

Cevap: Kaç kişiye ulaşacağı tahmin edilmiyor. Tahmin evet ya da hayır şeklindedir. Hoca kendi ifadesiyle öğrencinin hayatını kolaylaştırmaya çalıştığını, öğrencinin ise işi karmaşıklaştırdığını söyledi. Amaç sınıflandırıcıları web verisi olan bu video verisi üzerinde kullanmaktır.

Hoca demonun nasıl olması gerektiğini adım adım anlattı. Önce veri analizi süreci anlatılır:

- Aykırı veriler ayıklanır.
- Ki-kare testi ile birbirine bağlı öznitelikler çıkarılır.
- Normalizasyon yapılır.

- Üç makine öğrenmesi algoritması uygulanır.
- Veri %80 eğitim, %20 test olarak ayrılır.
- Üç modelin test verisi üzerindeki başarısı gösterilir: doğruluk, F-skor ve confusion matrix.

Buradan şu sonuç söylenebilir: Bu yöntemlerle bu veri setinde elde edilebilecek en yüksek başarı budur. Hocanın örneğiyle trend tahmininde en fazla %80'e çıkılabiliyor olabilir.

Ardından gerçek zamanlı uygulama gösterilir. Yeni bir videodan gerekli öznitelikler çıkarılır ve modeller etiket tahmin eder. Hoca şu senaryoyu önerdi: Geçen hafta 5 video için tahmin yapılmıştır. Uygulama birinci video için trend olacak, diğer 4 video için trend olmayacak demiştir. Bir hafta sonra gerçekte ne olduğuna bakılır. Sunum sırasında da o hafta yüklenmiş 4 yeni video için tahmin yapılır ve sonucu sonraki haftaya bırakılır.

Sonuç: Öğrencinin ekranda gösterdiği, trend etiketi olan hazır veri seti kullanılacaktır. Kullanılacak üç sınıflandırıcı adlarıyla yazılmalıdır. Uygulamak için hangi kütüphanenin kullanılacağı da belirtilmelidir. Hocanın örneği, Spark ML kütüphanesi ve LSTM için TensorFlow üzerinde Keras'tır.

14.7 Tüm Projeler İçin Ortak Beklentiler

Hoca konuşmalar boyunca dile getirdiği şartların bütün projeler için geçerli olduğunu söyledi. Bunlar şöyle toplanabilir:

- **Kitapla bağ:** Proje, Web Data Mining kitabındaki bir algoritmayı ya da bölümü kullanmalıdır.
- **Hazır veri seti:** Tarayıcı projeleri hariç hazır, indirilebilir bir veri seti kullanılmalıdır.
- **Veri setini açıklayan tablo:** Raporda yalnız link yetmez. Tabloda veri sayısı, etiketli bir veri ise hedef etiket, öznitelik sayısı, tweet veri setiyse tweet sayısı ve verinin toplanma zamanı bulunmalıdır.
- **Sınıflandırma projeleri:** En az 3 sınıflandırıcı (biri LSTM olabilir), kütüphane adları ve eğitim-test ayrımı.
- **Tarayıcı projeleri:** En az 3 farklı web kaynağı ve birleştirilmiş, hibrit gösterim.
- **Demo:** Bir kullanıcı arayüzü ve çalışan kod.
- **Boyut:** Proje iki ayda bitirilebilecek boyutta olmalı, gereksiz yere karmaşıklaştırılmamalıdır.

  Hocanın vurgusu: Proje önerisi çok net olmalıdır. "Şunu deneyeyim, bu oldu" türü bir öneri değil; hangi algoritmaların, hangi veri setinin kullanılacağı ve demoda neler gösterileceği açıkça yazılmalıdır.

  Ders sonrası not: Altyazıdaki bozuk ya da yanlış söylenmiş terimler bu notta düzeltilerek yazıldı. "Turn frequency" term frequency'dir. "Renk fonksiyonu" ranking function, "kosai similarity" cosine similarity'dir. "Future" feature, "CSS skoru" CVSS skorudur (altyazıda CVSS de geçiyor). "Natural Vulnerability Database" doğrusu National Vulnerability Database'dir. Şunlar ise altyazıdan doğrulanamadığı için tahmindir: "Natch" büyük olasılıkla Apache Nutch, "ISS server" büyük olasılıkla Microsoft IIS, "DLDB / DBL LP / BBLP" büyük olasılıkla DBLP, "CM" büyük olasılıkla SIEM, "word to act" büyük olasılıkla Word2Vec, "NERT" büyük olasılıkla adlandırılmış varlık tanıma (named entity recognition), "ITX Explorer" ve "ASM" büyük olasılıkla IEEE Xplore ve ACM, "Box2Scrap" büyük olasılıkla Books to Scrape, "Bing Scholar" büyük olasılıkla Microsoft Academic.

Dersin Toparlaması

Bu derste web içerik madenciliğinin bilgi erişimi tarafına girildi. Bilgi erişiminin yapısız dokümanlar ve yapısız sorgularla çalışması veri tabanı dünyasından ayrıldığı nokta olarak görüldü. Web aramasının, bilgi erişiminin büyük ölçekli ve dağıtık veriye özel hali olduğu ve köprü bağlantıları, tarama, hacim, güncellik, eş zamanlı kullanıcı ve spam gibi kendine özgü problemleri olduğu anlatıldı. Sonra bir modelin dört bileşeni incelendi: sorgu dili, sorgu gösterimi, doküman gösterimi, sıralama fonksiyonu. Boolean model kelimenin varlığına bakıp tam eşleşme ararken, vektör uzayı modeli TF-IDF ağırlıklarıyla ve kosinüs ya da Okapi gibi benzerlik ölçüleriyle sıralı sonuç üretir. İlgililik geri bildirimi de kullanıcının işaretlemeleriyle sorguyu genişletir.

Hoca bir sonraki konuyu da haber verdi. İki bilgi erişimi modeli, doküman ve sorgu gösterimi ile sıralama konuşulduktan sonra sıra, taranan dokümanlar üzerinde indekslemeden önce hangi ön işleme adımlarının yapıldığına (text preprocessing), sonra da indeks üzerindeki dokümanlarla kelimeler arasındaki benzerliğin bulunup sıralanmasına gelecektir. Dönem projeleri için de hoca, seçilen algoritma ve veri setiyle önerilerin netleştirilmesini istedi.

Not: İsterseniz bu kayıtları NotebookLM gibi bir araca atıp daha istifadeli hale getirebilirsiniz.